AN INTELLIGENT PUBLIC TRANSPORT DELAY PREDICTION AND ROUTE OPTIMIZATION SYSTEM

Technical Methodology and Flowchart Specification Report

PROJECT METHODOLOGY IMPLEMENTATION BLUEPRINT

Selected Objectives:	1. Multi-source ML delay prediction (NYC, Chicago baseline) 2. Route optimization via dynamic alternative paths & Explainable AI 3. Dual-perspective visual dashboard for commuters & transit authorities
Baseline Foundation:	GTFS timetable indexing, GPS trajectory logging, & OSM Graph routing
Novel Feature Set:	Schedule-Aware Delay Propagation & Explainable Dijkstradetours (X-MCDR)
Flowchart Model:	Fenced Mermaid configuration code embedded directly within report
Date of Publication:	June 2026

This blueprint details the custom basic and novel technical methodologies designed to execute your chosen project objectives. It translates baseline state-of-the-art transit models into a highly feasible, unique implementation roadmap with a structured Mermaid flowchart and code.

1. Basic Baseline Methodology

The basic baseline methodology represents the standard, state-of-the-art architecture for public transport delay prediction and routing. It is derived from established patterns in transit literature and common smart city implementations, structured as a linear data pipeline:

A. Data Ingestion & Preprocessing

- **General Transit Feed Specification (GTFS):** Loads schedules, including routes.txt, trips.txt, stops.txt, and stop_times.txt to establish planned arrival and departure timelines.
- **Operational Logs / Real-Time Feeds:** Obtains actual bus/train arrival and departure times (e.g. historical AVL logs from NYC MTA or Chicago CTA). Delays are calculated as: Delay = Actual Time - Scheduled Time.
- **Exogenous Data Fusion:** Merges coordinates with meteorological reports (hourly rainfall, visibility) and calendar databases (national holidays, peak commute hours).
- **Cleaning & Data Imputation:** Standardizes timestamps, cleans outliers, and handles missing sensor values (using historical segment averages or basic KNN imputation).

B. Machine Learning Delay Modeling

- **Classification Tasks:** Predicts binary delay outcomes (e.g. is_delayed = 1 if delay > 5 minutes else 0) using algorithms like Random Forest or XGBoost Classifier. Performance is evaluated via ROC-AUC and F1-score.
- **Regression Tasks:** Predicts the exact delay duration in minutes using Gradient Boosting Regressors or LightGBM. Evaluated via Mean Absolute Error (MAE) and RMSE.

C. Baseline Route Optimization

- **OSM Graph Representation:** Extracts the street network using OpenStreetMap (OSM) via OSMnx, where intersections represent nodes and street segments represent edges.
- **Static Pathfinding:** Calculates routes using Dijkstra or A* algorithms weighted by static distance or estimated average speed, without incorporating real-time delay variations.

D. Interface Visualization

- **Interactive Dashboard:** Builds a basic Streamlit dashboard featuring cards for key metrics (Average Delay, Most Delayed Route) and Plotly charts showing average delay by hour and route.

2. Unique and Novel Methodology

To distinguish your final-year project and achieve academic excellence, we introduce a unique and novel methodology that remains highly feasible for a B.Tech timeline. This methodology directly addresses the 'black-box' and 'isolated model' limitations identified in the research literature:

Innovation 1: Schedule-Aware Sequential Delay Propagation (SA-SDP)

- Concept: Instead of treating stops as isolated data records, the system models delays as a sequential cascade.
- Feasible Implementation: Rather than using complex Graph Neural Networks (GNNs), we implement a Schedule-Aware Sequential feature framework. When predicting the delay at Stop (i), the actual or predicted delay of the vehicle at the previous stop (i-1) is injected as a dynamic feature.
- Advantage: Accurately models cascading delay propagation across route segments using standard, high-performance tabular algorithms (XGBoost/LightGBM) without GNN training overhead.

Innovation 2: Explainable Multi-Criteria Dijkstra Routing (X-MCDR)

- Concept: Combines dynamic pathfinding with Explainable AI (XAI) to generate transparent routing recommendations.
- Dynamic Weighting: OpenStreetMap (OSM) edge weights are dynamically updated: Edge Weight = Base Travel Time + Predicted Delay Minutes.
- Transparent Detours: If the routing engine suggests a detour, it queries the SHAP (Shapley Additive exPlanations) engine for the delayed segment. The dashboard then displays a natural-language reason: 'Route 1 is bypassed because Stop B has an 82% delay probability (contributing +15 mins) driven by high traffic volume (SHAP +8 min) and heavy rain (SHAP +5 min).'
- Advantage: Solves the 'black-box' gap in transit routing, building passenger and operator trust.

Innovation 3: Dual-Perspective Decision Support Architecture

- Concept: Tailors the interactive dashboard to serve two distinct user classes.
- Commuter Panel: Provides live delay predictions, optimal alternative routing, and explainable detour cards.
- Transit Planner Panel: Applies spatial density clustering (DBSCAN) on coordinates of persistent delays to identify structural physical bottlenecks (hotspots). It generates automated recommendations for planners: 'Route 204: Delay probability at Stop A exceeds 75% between 8-9 AM. Action: Inject 5 min buffer time or add one extra vehicle.'

3. Methodology Flowchart & Mermaid Code

Below is the raw, copy-pasteable Mermaid syntax representing your project's methodology flowchart. You can copy this code and paste it into any Mermaid-compatible viewer (such as GitHub, Notion, Mermaid Live Editor, or draw.io) to generate the high-resolution diagram.

> **Mermaid Flowchart Syntax**

```
graph TD
    %% Data Ingestion and Fusion
    subgraph Data Ingestion & Preprocessing
        A[GTFS Static Schedules] --> D[Multimodal Data Fusion Engine]
        B[Real-Time Transit Feeds / Historical Logs] --> D
        C[Exogenous Data: weather, traffic, holidays] --> D
        D --> E[Data Cleaning & KNN Imputation]
        E --> F[Feature Engineering: Temporal, Spatial, Delay Propagation]
    end

    %% Modeling and Explainability
    subgraph Predictive Modeling & XAI
        F --> G[Explainable ML Delay Predictor]
        G --> H1[XGBoost Classifier: Delay Prob]
        G --> H2[LightGBM Regressor: Delay Mins]
        H1 & H2 --> I[SHAP Explainability Engine]
        I --> J[Local & Global Feature Interpretations]
    end

    %% Spatial Clustering & Hotspots
    subgraph Geospatial Bottleneck Detection
        F --> K[DBSCAN Clustering Engine]
        K --> L[Geospatial Delay Hotspots Map]
    end

    %% Route Optimization
    subgraph Dynamic Route Optimization
        H2 --> M[Dynamic Delay-Weighted Routing Engine]
        N[OpenStreetMap Road Network] --> M
        M --> O[Explainable Multi-Criteria Dijkstra Routing X-MCDR]
    end

    %% UI Dashboard
    subgraph Interactive Smart City Dashboard
        O & J & L --> P[Interactive Dashboard]
        P --> Q1[Commuter View: Delay Predictions & Explanations]
        P --> Q2[Transit Authority View: Hotspots & Recommendations]
    end

    style D fill:#1E3A8A,stroke:#3B82F6,stroke-width:2px,color:#fff
    style G fill:#1E3A8A,stroke:#3B82F6,stroke-width:2px,color:#fff
    style K fill:#1E3A8A,stroke:#3B82F6,stroke-width:2px,color:#fff
    style M fill:#1E3A8A,stroke:#3B82F6,stroke-width:2px,color:#fff
    style P fill:#1E3A8A,stroke:#3B82F6,stroke-width:2px,color:#fff
```

Flowchart Stage Descriptions

- **1. Ingestion & Fusion:** Merges GTFS, transit feeds, traffic logs, weather, and holidays into a clean data frame with KNN imputation.
- **2. Modeling & XAI:** Trains classification (XGBoost) and regression (LightGBM) models, then utilizes SHAP to extract global and local explainability values.
- **3. Hotspots Map:** Uses DBSCAN clustering on persistent delay coordinates to identify and map geographic bottlenecks.

- **4. Routing Engine:** Updates OSM street graph weights using predicted delay durations, then finds the Dijkstra weighted by delay times (X-MCDR).
- **5. Dashboards:** Integrates commuter detour cards with explanations and transit operator buffer/fleet adjustments in Streamlit.